

Road Detection Using CNN and U-Net Segmentation

Deep Learning Image Segmentation Project

Project Documentation

Environment: Google Colab + TensorFlow/Keras + KaggleHub

Dataset: [mariahadjmessaoud/road-detection-dataset](#)

Prepared from the submitted Colab notebook

Table of Contents

1. Executive Summary
2. Project Introduction
3. Problem Statement
4. Project Objectives
5. Dataset Description
6. Dataset Structure and Exploratory Analysis
7. Data Preprocessing
8. Train/Validation/Test Split
9. Data Generator and Batch Processing
10. CNN / U-Net Architecture
11. Loss Functions and Evaluation Metrics
12. Model Compilation and Training
13. Training Results
14. Test Evaluation
15. Road Mask Prediction and Visualization
16. Model Saving and Deployment Workflow
17. Strengths of the Project
18. Limitations and Improvement Opportunities
19. Conclusion
20. Future Work
21. Technical Specifications
22. Appendix: Key Code Components

1. Executive Summary

This project implements an automated road detection system using deep learning. The task is formulated as binary semantic image segmentation: for every pixel in a road image, the model predicts whether that pixel belongs to the road region or the background. The submitted notebook uses TensorFlow/Keras and a U-Net-style convolutional neural network.

The project uses the Kaggle dataset 'mariahadjmessaoud/road-detection-dataset'. The training portion contains 5,442 image-mask pairs. Original images are $512 \times 512 \times 3$, while their corresponding grayscale masks are $512 \times 512 \times 1$. For efficient model training, both are resized to $256 \times 256 \times 3$.

The notebook splits the 5,442 training pairs into 4,353 training samples, 544 validation samples, and 545 test samples. The CNN was trained for 10 epochs with batch size 16, Adam optimization, and a learning rate of 1×10^{-4} .

On the recorded test evaluation, the model achieved 97.24% pixel accuracy, a Dice coefficient of 0.6395, and IoU of 0.5041, with test loss 0.4638. Because this is a segmentation task, Dice and IoU are particularly important alongside accuracy.

2. Project Introduction

Road detection is an important computer vision task for intelligent transportation, advanced driver-assistance systems, autonomous vehicles, road-scene understanding, traffic analysis, and mapping. Instead of assigning one label to an entire image, semantic segmentation produces a pixel-level road mask.

The project therefore uses a segmentation-oriented CNN rather than a conventional image classification network. The U-Net design combines deep feature extraction with skip connections that preserve spatial information needed to recover road boundaries.

3. Problem Statement

Given a road-scene image, the system must identify the pixels that represent road areas and produce a binary segmentation mask. The input is a three-channel RGB image and the output is a one-channel probability mask.

Input	Output	Task Type
RGB road image	Binary road mask	Semantic image segmentation

4. Project Objectives

- Download and inspect the road detection dataset in Google Colab.
- Identify matching image and mask files.
- Load RGB images and grayscale segmentation masks.
- Resize images and masks to a fixed model input size.
- Normalize image and mask values.
- Build a batch-based TensorFlow data generator.
- Develop a U-Net-style CNN for binary road segmentation.
- Use Dice loss together with Binary Cross-Entropy.
- Monitor accuracy, Dice score, IoU, and loss during training.
- Use EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint callbacks.
- Evaluate the trained model on a held-out test set.
- Visualize predicted road masks and road overlays.
- Save the trained model for later inference.

5. Dataset Description

The notebook downloads the dataset using KaggleHub with the identifier 'mariahadjmessaoud/road-detection-dataset'. The detected dataset directory is 'Final_Challenge_Dataset_512'. The dataset contains separate image and mask directories for train, public_test, and private_test.

Dataset component	Observed structure
Root	Final_Challenge_Dataset_512
Training	train/images and train/masks
Public test	public_test/images and public_test/masks
Private test	private_test/images and private_test/masks
Training image-mask pairs used	5,442
Image format observed	.png
Image size observed	512 × 512 × 3
Mask size observed	512 × 512 × 1

The notebook confirms that image and mask filenames correspond directly. For example, road_0001.png appears in both the training image and mask directories. This filename matching is important because each image must be paired with its correct ground-truth mask.

6. Dataset Structure and Exploratory Analysis

The notebook first walks through the downloaded directory tree to discover the available folders. A recursive file search initially finds 14,084 image files across the dataset because the search includes train and test areas. The project then deliberately selects the training images and masks for supervised model development.

After selecting the training directories, the notebook reports exactly 5,442 images and 5,442 masks. A sample image and corresponding grayscale mask are loaded and visualized. Multiple image-mask examples are also displayed to verify the data before training.

7. Data Preprocessing

1. Image loading: RGB images are loaded with three channels.
2. Mask loading: masks are loaded in grayscale with one channel.
3. Resizing: images and masks are resized from 512×512 to 256×256 for model training.
4. Normalization: RGB image values are divided by 255 so they fall approximately in the range 0–1.
5. Mask normalization: mask values are divided by 255 and thresholded at 0.5 to form a binary mask.
6. Batch preparation: samples are grouped into batches of 16.

The preprocessing pipeline produces batches with image shape (16, 256, 256, 3) and mask shape (16, 256, 256, 1), as recorded in the notebook.

8. Train/Validation/Test Split

The 5,442 supervised training pairs are divided using a two-stage train_test_split procedure. First, 20% is held out as a temporary set. That temporary set is then split equally into validation and test sets.

Subset	Samples	Approx. percentage
Training	4,353	80%
Validation	544	10%

Subset	Samples	Approx. percentage
Test	545	10%
Total	5,442	100%

A fixed `random_state` of 42 is used so the split can be reproduced.

9. Data Generator and Batch Processing

A custom `RoadDataGenerator` class extends `tf.keras.utils.Sequence`. It loads image-mask pairs only when batches are requested instead of loading the entire dataset into memory at once. This is useful for image segmentation because image tensors can consume substantial memory.

The generator supports shuffling for training data and disables shuffling for validation and test data. The recorded batch size is 16.

10. CNN / U-Net Architecture

The model is a U-Net-style convolutional neural network. The encoder progressively reduces spatial resolution while increasing the number of feature channels. The bottleneck captures high-level contextual information. The decoder restores spatial resolution and combines decoder features with encoder features through skip connections.

Stage	Main operation	Filters
Encoder 1	Conv block + MaxPooling + Dropout	32
Encoder 2	Conv block + MaxPooling + Dropout	64
Encoder 3	Conv block + MaxPooling + Dropout	128
Encoder 4	Conv block + MaxPooling + Dropout	256
Bottleneck	Conv block + Dropout	512
Decoder 1	Transpose convolution + skip + Conv block	256
Decoder 2	Transpose convolution + skip + Conv block	128
Decoder 3	Transpose convolution + skip + Conv block	64
Decoder 4	Transpose convolution + skip + Conv block	32
Output	1×1 Conv2D + sigmoid	1

Each convolutional block contains two 3×3 convolution layers, Batch Normalization, and ReLU activation. MaxPooling performs downsampling in the encoder, while Conv2DTranspose performs upsampling in the decoder. The final sigmoid layer outputs a per-pixel probability between 0 and 1.

11. Loss Functions and Evaluation Metrics

11.1 Binary Cross-Entropy

Binary Cross-Entropy measures the difference between the ground-truth binary mask and the predicted probability mask. It provides a pixel-level classification loss.

11.2 Dice Coefficient

The Dice coefficient measures overlap between the predicted road region and the ground-truth road region. The notebook defines Dice directly and uses $1 - \text{Dice}$ as Dice loss.

11.3 Combined Loss

The training loss is the mean Binary Cross-Entropy plus Dice loss. This combination encourages both pixel-wise correctness and overlap of the predicted road region.

11.4 Intersection over Union (IoU)

IoU measures the intersection between predicted and true road regions divided by their union. It is a standard segmentation metric and is more informative than raw pixel accuracy when the background occupies a large portion of the image.

11.5 Pixel Accuracy

Pixel accuracy reports the fraction of pixels classified correctly. In segmentation, accuracy should be interpreted together with Dice and IoU because a model can obtain high accuracy by predicting large background regions correctly.

12. Model Compilation and Training

Training setting	Value
Framework	TensorFlow 2.21.0
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	16
Epochs	10
Input size	$256 \times 256 \times 3$
Output	1-channel sigmoid mask
Loss	Binary Cross-Entropy + Dice Loss
Checkpoint	best_road_segmentation_model.keras

Three callbacks are used. EarlyStopping monitors validation loss and can restore the best weights.

ReduceLROnPlateau reduces the learning rate when validation loss stops improving. ModelCheckpoint saves the best model according to validation loss.

13. Training Results

The notebook records 10 completed epochs. Validation loss improved through epoch 7, temporarily worsened during epochs 8–9, and improved again at epoch 10. The best recorded validation loss at the end of the run was 0.46437.

Epoch	Train Acc.	Val. Acc.	Train Dice	Val. Dice	Val. IoU	Val. Loss
1	0.9660	0.9642	0.5279	0.5375	0.4326	0.5747
2	0.9673	0.9695	0.5521	0.5698	0.4636	0.5292
3	0.9683	0.9665	0.5712	0.5786	0.4634	0.5265
4	0.9691	0.9719	0.5868	0.5819	0.4612	0.5175
5	0.9696	0.9691	0.5977	0.6088	0.4829	0.4906
6	0.9702	0.9719	0.6058	0.6126	0.4884	0.4801
7	0.9709	0.9712	0.6169	0.6254	0.4955	0.4695
8	0.9714	0.9701	0.6266	0.6124	0.4817	0.4846
9	0.9719	0.9736	0.6355	0.6231	0.4847	0.4746
10	0.9726	0.9728	0.6453	0.6319	0.4954	0.4644

The training accuracy increased from approximately 96.60% at epoch 1 to 97.26% at epoch 10. Validation accuracy reached 97.36% at epoch 9 and was 97.28% at epoch 10. Validation Dice improved from 0.5375 to 0.6319.

14. Test Evaluation

Metric	Recorded test value
Test loss	0.4638
Pixel accuracy	97.24%
Dice coefficient	0.6395
IoU	0.5041

The test generator contains 545 samples. The notebook reports 35 test batches and produces a prediction tensor of (545, 256, 256, 1). The recorded pixel accuracy is high, but the lower Dice and IoU values show why segmentation quality should not be judged from accuracy alone.

The test metrics are based on the model state restored after training. The notebook's evaluation output labels the aggregate metric container as 'compile_metrics', while the batch evaluation stream separately reports accuracy, Dice, IoU, and loss.

15. Road Mask Prediction and Visualization

After evaluation, the model predicts masks for the test set. For visualization, the predicted probability mask is thresholded at 0.5 to produce a binary road mask.

The notebook displays three views: the original image, the true road mask, and the predicted road mask. It also creates an overlay by placing the predicted mask on top of the original road image.

16. Model Saving and Deployment Workflow

The trained model is saved in Keras format as 'road_detection_cnn.keras'. A separate inference workflow can upload a new road image, convert it to RGB, resize it to 256×256, normalize it, add a batch dimension, and pass it to `model.predict()`.

The predicted output is thresholded at 0.5 and visualized as a road mask or an overlay.

17. Strengths of the Project

- The task is correctly formulated as semantic segmentation rather than ordinary image classification.
- The dataset contains paired road images and ground-truth masks.
- The U-Net architecture is appropriate for recovering pixel-level spatial boundaries.
- Batch Normalization improves the stability of convolutional blocks.
- Dropout is included to reduce overfitting risk.
- Combined Binary Cross-Entropy and Dice loss addresses both pixel-wise error and region overlap.
- IoU and Dice are reported in addition to accuracy.
- Validation monitoring and model checkpointing are included.
- The notebook includes visual comparison of original, true mask, and predicted mask.
- The final model is saved for future inference.

18. Limitations and Improvement Opportunities

- Use data augmentation such as horizontal flipping, small rotations, zooming, and brightness variation.
- Experiment with a larger input size, for example 512×512, if GPU memory permits.
- Train for more epochs with the existing EarlyStopping mechanism when additional training time is available.
- Experiment with Dice-Focal or weighted losses when road/background imbalance is strong.
- Tune the mask threshold instead of assuming 0.5 is optimal.

- Add precision, recall, F1/Dice, and class-wise pixel statistics for more detailed evaluation.
- Compare the U-Net baseline with U-Net variants or encoder backbones using transfer learning.
- Use morphological post-processing to remove small isolated false-positive regions if required.
- Evaluate separately on public and private test subsets when ground truth is intended for evaluation.
- Use data augmentation consistently on image-mask pairs so that the geometric transformation applied to an image is also applied to its mask.

19. Conclusion

This project successfully builds a CNN-based road segmentation pipeline in Google Colab. The workflow covers dataset acquisition, directory inspection, paired image-mask loading, preprocessing, train/validation/test splitting, batch generation, U-Net model construction, custom loss functions, training callbacks, evaluation, visualization, and model saving.

The recorded test performance of 97.24% pixel accuracy, 0.6395 Dice, and 0.5041 IoU demonstrates that the model learned meaningful road-region representations. The project also demonstrates that high pixel accuracy alone does not fully describe segmentation quality, so overlap-based metrics such as Dice and IoU should be reported.

20. Future Work

1. Introduce stronger image-mask augmentation.
2. Increase resolution and train with multi-scale inputs.
3. Experiment with pretrained encoders and transfer learning.
4. Benchmark U-Net against other segmentation architectures.
5. Tune hyperparameters using a validation protocol.
6. Create a lightweight inference application for uploaded images or video frames.
7. Extend the system from static images to real-time road-scene video segmentation.
8. Evaluate robustness under night, rain, fog, shadows, and unusual road conditions.

21. Technical Specifications

Category	Specification
Platform	Google Colab
Hardware recorded	NVIDIA T4 GPU environment in notebook metadata
Python ecosystem	TensorFlow/Keras, NumPy, Pandas, Matplotlib, scikit-learn, PIL
TensorFlow	2.21.0
Dataset source	Kaggle via KaggleHub
Dataset ID	mariahadjmessaoud/road-detection-dataset
Image type	PNG RGB
Mask type	PNG grayscale
Original resolution	512 × 512
Training resolution	256 × 256
Training samples	4,353
Validation samples	544

Category	Specification
Test samples	545
Batch size	16
Epochs	10
Optimizer	Adam
Learning rate	1×10^{-4}
Model	U-Net-style CNN
Output activation	Sigmoid
Saved model	road_detection_cnn.keras

22. Appendix: Key Code Components

Dataset download

```
import kagglehub
path = kagglehub.dataset_download(
    "mariahadjmessaoud/road-detection-dataset"
)
```

Preprocessing

```
image = image / 255.0
mask = mask / 255.0
mask = (mask > 0.5).astype(np.float32)
```

Model output

```
outputs = layers.Conv2D(
    1, 1, padding="same", activation="sigmoid"
)(d4)
```

Compilation

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=combined_loss,
    metrics=["accuracy", dice_coefficient, iou_metric]
)
```

Training

```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=callbacks
)
```

Saving

```
model.save("road_detection_cnn.keras")
```